Customizing Your Specs' Output

When you use RSpec on a real-world project, you'll build up a suite of dozens, hundreds, or even thousands of examples. Most test frameworks, including RSpec, are optimized for this kind of use. The default output format hides a lot of detail so that it can show your specs' progress.

The Progress Formatter

In this section, we're going to look at a different ways to view your specs' output. Create a new file called spec/coffee_spec.rb with the following contents:

02-running-specs/01/spec/coffee_spec.rb

```
RSpec.describe 'A cup of coffee' do
  let(:coffee) { Coffee.new }

  it 'costs $1' do
    expect(coffee.price).to eq(1.00)
  end

➤ context 'with milk' do
    before { coffee.add :milk }

    it 'costs $1.25' do
      expect(coffee.price).to eq(1.25)
    end
  end
end
```

This spec file uses the same techniques we saw in the previous chapter, with one new twist: the context block starting on the highlighted line. This method groups a set of examples and their setup code together with a common description—in this case "with milk." You can nest these example groups as deeply as you want.

There's nothing mysterious going on behind the scenes here: context is just an alias for describe. You could use them interchangeably, but we tend to use context for phrases that modify the object we're testing, the way "with milk" modifies "A cup of coffee."

This spec will need a Coffee class to test. In a full project, you'd put its definition in a separate file and use require in your specs. But for this simple example, it's fine just to put the class at the top of your spec file. Here's the start of an implementation that's not quite enough to pass the specs yet:

02-running-specs/01/spec/coffee_spec.rb

```
class Coffee
  def ingredients
    @ingredients ||= []
  end
```